**Bangladesh University of Business and Technology**
Department of Computer Science & Engineering

**Machine Learning Lab (CSE 403 / CSE 404)**

# LAB 04

## Data Preprocessing in Machine Learning

— Titanic Survival Dataset —

Bijon Mallik | Lecturer, Dept. of CSE
Intake 52 • Section 06 & 07
Duration: 2.5 hours | Level: Beginner–Intermediate

## 0. Lab Overview

### Learning Objectives

- Understand the Titanic dataset structure and identify real-world data quality issues
- Load, inspect, and perform EDA on a CSV dataset using Pandas
- Handle missing values using appropriate imputation strategies
- Detect and treat outliers using IQR and visualisations
- Engineer new features from existing columns (FamilySize, Title)
- Encode categorical variables without introducing ordinal bias
- Scale numerical features to remove distance/gradient distortions
- Split data correctly with stratification on the target class
- Build a leak-free scikit-learn ColumnTransformer + Pipeline
- Train and compare Logistic Regression vs Random Forest on 5 metrics

### Lab Schedule

| Topic | Techniques Covered | Est. Time |
|---|---|---|
| Dataset & EDA | Load CSV, isnull, describe, value_counts, class balance | 20 min |
| Missing Data | Mean / Median / Mode imputation, drop column | 20 min |
| Outliers | Box plots, IQR, Winsorising, RobustScaler rationale | 15 min |
| Feature Engineering | FamilySize, Title extraction from Name | 15 min |
| Feature Selection | Correlation heatmap, drop irrelevant columns | 15 min |

| | | |
|---|---|---|
| Encoding | Label Encoding (Sex), One-Hot (Embarked) | 15 min |
| Scaling | MinMax vs Standard vs Robust on Age, Fare | 15 min |
| Pipeline | ColumnTransformer + Pipeline, Joblib export | 20 min |
| Model & Eval | LogReg vs Random Forest, 5 metrics, AUC-ROC | 20 min |

## Prerequisites

- Lab 01: Python basics — NumPy, Pandas
- Lab 02: Data visualisation — Matplotlib, Seaborn
- Lab 03: Supervised Learning foundations

# 1. Meet the Dataset — Titanic

The Titanic dataset describes 891 passengers of the RMS Titanic. Each row is one passenger. The goal is to predict whether a passenger **survived (1) or died (0)** the disaster — a binary classification problem.

## 1.1 How to Load the Dataset

### Option A — Direct URL (recommended for lab, no login needed)

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

url =
'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
df = pd.read_csv(url)
print(df.shape)       # (891, 12)
df.head()
```

### Option B — Seaborn built-in (zero setup)

```
df = sns.load_dataset('titanic')
df.head()
```

### Option C — Kaggle download

Go to https://www.kaggle.com/c/titanic/data → Data tab → Download train.csv

## 1.2 Column Reference

| Column | Full Name | Type | Description |
|---|---|---|---|
| PassengerId | Passenger ID | Numeric | Serial number only — no predictive value; DROP |
| **Survived** | **Survival** | **Binary** | **TARGET: 0 = Died, 1 = Survived** |
| Pclass | Passenger Class | Ordinal | Ticket class — 1st (rich), 2nd, 3rd (poor) |
| Name | Full Name | Text | Contains title (Mr., Mrs., Miss., Master.) |
| Sex | Sex | Categorical | male or female — needs Label Encoding |
| Age | Age (years) | Numeric | ~20% missing values — needs imputation |
| SibSp | Siblings/Spouses | Numeric | # of siblings or spouses aboard |
| Parch | Parents/Children | Numeric | # of parents or children aboard |
| Ticket | Ticket Number | Text | Mostly unique — DROP (low signal) |
| Fare | Fare (£) | Numeric | Ticket price; right-skewed, has outliers |

| | | | |
|---|---|---|---|
| Cabin | Cabin Number | Text | ~77% missing — DROP or binarise |
| Embarked | Port of Boarding | Categorical | S=Southampton, C=Cherbourg, Q=Queenstown |

## 1.3 Initial EDA

```
# Shape and data types
print(df.shape)          # rows, columns
print(df.dtypes)
print(df.describe())     # stats for numerical columns

# Missing value audit
missing = df.isnull().sum().sort_values(ascending=False)
pct     = (missing / len(df)) * 100
print(pd.concat([missing, pct], axis=1, keys=['Count', '%']))

# Class balance — how many survived?
print(df['survived'].value_counts())
print(df['survived'].value_counts(normalize=True).round(3))
# Output: 0 → 61.6%  (died)   1 → 38.4%  (survived)

# Missing value heatmap
plt.figure(figsize=(10, 5))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False)
plt.title('Missing Data Heatmap — Titanic'); plt.tight_layout(); plt.show()
```

**Expected Missing Values:** Age: ~177 rows (20%) | Cabin: ~687 rows (77%) | Embarked: 2 rows (<1%)

# 2. Handling Missing Data

The Titanic dataset has three columns with missing values. Each requires a different treatment strategy.

## 2.1 Missingness Summary

| Column | % Missing | Strategy | Reason |
|---|---|---|---|
| age | ~20% | Median imputation | Numerical, right-skewed — median robust to outliers |
| cabin | ~77% | Drop column | Too many nulls; binarise to has_cabin=0/1 optionally |
| embarked | <1% | Mode imputation | Categorical; only 2 rows missing — fill with 'S' |

## 2.2 Imputation Code

```
from sklearn.impute import SimpleImputer

# ── Step 1: Drop cabin (>77% missing)
df = df.drop(columns=['cabin'])

# ── Step 2: Median impute 'age' (numerical, skewed)
imp_med = SimpleImputer(strategy='median')
df[['age']] = imp_med.fit_transform(df[['age']])  # fit+transform on full df for
EDA
# IMPORTANT: In the pipeline, fit ONLY on X_train!

# ── Step 3: Mode impute 'embarked' (categorical, 2 rows)
imp_mode = SimpleImputer(strategy='most_frequent')
df[['embarked']] = imp_mode.fit_transform(df[['embarked']])

# Verify — should show zeros
print(df[['age','embarked']].isnull().sum())
```

**Key Rule:** Always fit imputers on TRAINING data only — never on full df before splitting. The code above is for EDA exploration only. In Section 8, imputers live inside the Pipeline.

# 3. Detecting & Treating Outliers

The fare column is heavily right-skewed with a few extreme values (max ~$512). The age column also has some outliers. We detect them visually and with IQR, then treat with Winsorising or RobustScaler.

## 3.1 Visual Detection — Box Plots

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.boxplot(y=df['fare'],  ax=axes[0], color='steelblue')
axes[0].set_title('Fare Distribution — Outliers Visible')

sns.boxplot(y=df['age'],   ax=axes[1], color='coral')
axes[1].set_title('Age Distribution')

plt.tight_layout(); plt.show()
```

## 3.2 IQR Detection

```
for col in ['fare', 'age']:
    Q1  = df[col].quantile(0.25)
    Q3  = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower  = Q1 - 1.5 * IQR
    upper  = Q3 + 1.5 * IQR
    n_out  = df[(df[col] < lower) | (df[col] > upper)].shape[0]
    print(f'{col}: {n_out} outliers (lower={lower:.1f}, upper={upper:.1f})')
```

## 3.3 Treatment — Winsorising

```
from scipy.stats import mstats

# Cap top and bottom 1% for fare
df['fare'] = mstats.winsorize(df['fare'], limits=[0.01, 0.01])

# Verify
print(df['fare'].describe())
```

**Note:** Tree-based models (Random Forest, XGBoost) are naturally outlier-tolerant — scaling or Winsorising is mainly needed for linear models, KNN, and SVM.

# 4. Feature Engineering

Feature engineering creates new informative columns from existing ones. Two are particularly powerful in Titanic:

## 4.1 FamilySize

Combine SibSp (siblings/spouses) and Parch (parents/children) into a single FamilySize feature. Solo travellers had different survival rates from families.

```
# FamilySize = self + siblings/spouses + parents/children
df['family_size'] = df['sibsp'] + df['parch'] + 1

# Also create a binary 'alone' flag
df['is_alone'] = (df['family_size'] == 1).astype(int)

# Check survival rate by family size
print(df.groupby('family_size')['survived'].mean().round(2))
```

## 4.2 Title Extraction from Name

The Name column contains a title (Mr., Mrs., Miss., Master., Dr., etc.) which captures age, gender, and social status in one signal.

```
# Extract title using regex
df['title'] = df['name'].str.extract(r',\s*([^.]+)\.', expand=False).str.strip()

# Consolidate rare titles
rare =
['Rev','Dr','Col','Major','Capt','Jonkheer','Don','Sir','Lady','Countess','Dona']
df['title'] = df['title'].replace(rare, 'Rare')
df['title'] = df['title'].replace({'Mlle': 'Miss', 'Ms': 'Miss', 'Mme': 'Mrs'})

# Survival rate by title
print(df.groupby('title')['survived'].mean().sort_values(ascending=False).round(2))
# Master (young boys) and Mrs have highest survival rates
```

## 4.3 Optional — has_cabin

```
# Was cabin info recorded? (proxy for 1st class)
df['has_cabin'] = df['cabin'].notna().astype(int)    # run BEFORE dropping cabin
```

**Drop after engineering:** After creating these features, drop: name, ticket, passengerid, cabin, sibsp, parch (they are now captured in family_size/title).

# 5. Feature Selection

After engineering, decide which columns to keep. Three quick methods:

## 5.1 Drop Irrelevant Columns

```
drop_cols = ['passengerid', 'name', 'ticket', 'cabin', 'sibsp', 'parch']
df = df.drop(columns=[c for c in drop_cols if c in df.columns])
print(df.columns.tolist())
# Remaining: survived, pclass, sex, age, fare, embarked,
#             family_size, is_alone, title, has_cabin
```

## 5.2 Correlation Heatmap (Numerical Features)

```
plt.figure(figsize=(8, 6))
corr = df.select_dtypes(include='number').corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            linewidths=0.5, square=True)
plt.title('Correlation Heatmap — Titanic Numerical Features')
plt.tight_layout(); plt.show()

# Look for: high correlation with 'survived' → keep
#           high correlation between features → consider dropping one
```

## 5.3 Survival Rate by Category (Quick Filter)

```
for col in ['sex', 'pclass', 'embarked', 'title']:
    print(f'\n--- {col} ---')

print(df.groupby(col)['survived'].mean().sort_values(ascending=False).round(3))
```

**Rule of thumb:** Keep features that show meaningful variation in survival rate. Features that show identical survival rates across all categories add noise.

# 6. Encoding Categorical Features

After engineering, the dataset has 4 categorical columns. Each needs a different encoding strategy.

## 6.1 Encoding Strategy per Column

| Column | Strategy & Reason |
|---|---|
| **sex** | Label Encoding → male=0, female=1 (binary, no false order issue) |
| **embarked** | One-Hot Encoding → 3 categories, no natural order, low cardinality |
| **pclass** | Keep as-is (already ordinal 1 < 2 < 3) or OrdinalEncoder |
| **title** | One-Hot Encoding → Mr, Mrs, Miss, Master, Rare (5 values) |

## 6.2 Manual Encoding (Exploration)

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Label encode 'sex'
le = LabelEncoder()
df['sex_enc'] = le.fit_transform(df['sex'])   # male=1, female=0
print(dict(zip(le.classes_, le.transform(le.classes_))))

# One-Hot encode 'embarked' via pandas
df = pd.get_dummies(df, columns=['embarked'], prefix='emb', drop_first=True)

# One-Hot encode 'title' via pandas
df = pd.get_dummies(df, columns=['title'], prefix='title', drop_first=True)

print(df.shape)   # see new columns added
```

**Watch Out!** Use drop_first=True in get_dummies to drop one dummy column and avoid multicollinearity (the dummy variable trap) — important for linear models.

# 7. Feature Scaling

Two numerical features need scaling before feeding into linear models or KNN: age and fare. Tree-based models (Random Forest) do NOT require scaling but it never hurts.

## 7.1 Which Scaler to Use?

| Scaler | When to Use for Titanic |
|---|---|
| **StandardScaler** | age — approximately bell-shaped; good for Logistic Regression, SVM |
| **RobustScaler** | fare — has outliers; uses median+IQR, not affected by extremes |
| **MinMaxScaler** | Either column when you need [0,1] range (e.g., neural nets) |

## 7.2 Scaling Code with Before/After Visualisation

```
from sklearn.preprocessing import StandardScaler, RobustScaler, MinMaxScaler

# Compare scalers on 'fare'
fare = df[['fare']].copy()
scalers = {
    'Original':    fare['fare'],
    'StandardScaler': StandardScaler().fit_transform(fare).ravel(),
    'RobustScaler':   RobustScaler().fit_transform(fare).ravel(),
    'MinMaxScaler':   MinMaxScaler().fit_transform(fare).ravel(),
}

fig, axes = plt.subplots(1, 4, figsize=(14, 4))
for ax, (name, vals) in zip(axes, scalers.items()):
    ax.hist(vals, bins=40, color='steelblue', edgecolor='white')
    ax.set_title(name, fontsize=9)
plt.suptitle('Fare Distribution — Scaler Comparison')
plt.tight_layout(); plt.show()
```

**Critical Rule:** ALWAYS fit scalers on X_train ONLY. Apply the fitted scaler to X_val and X_test. Fitting on full dataset before splitting is data leakage — the most common student mistake.

# 8. Building a Leak-Free Pipeline

A scikit-learn Pipeline chains imputation, encoding, scaling, and the model into one object. It ensures all transformations are fitted only on training data — eliminating data leakage.

## 8.1 Prepare Features and Target

```
# Use the Seaborn version (has clean column names)
df_raw = sns.load_dataset('titanic')

# Feature engineering first
df_raw['family_size'] = df_raw['sibsp'] + df_raw['parch'] + 1
df_raw['is_alone']    = (df_raw['family_size'] == 1).astype(int)
df_raw['title']       = df_raw['name'].str.extract(r',\s*([^.]+)\.',
expand=False).str.strip()
rare =
['Rev','Dr','Col','Major','Capt','Jonkheer','Don','Sir','Lady','Countess','Dona']
df_raw['title'] = df_raw['title'].replace(rare, 'Rare')
df_raw['title'] =
df_raw['title'].replace({'Mlle':'Miss','Ms':'Miss','Mme':'Mrs'})

# Select final feature set
FEATURES =
['pclass','sex','age','fare','embarked','family_size','is_alone','title']
TARGET   = 'survived'

X = df_raw[FEATURES]
y = df_raw[TARGET]

# Stratified train-test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
print(f'Train: {X_train.shape}  |  Test: {X_test.shape}')
```

## 8.2 Build the Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, RobustScaler, OneHotEncoder,
OrdinalEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

num_cols = ['age', 'fare', 'family_size', 'is_alone']
cat_ohe  = ['sex', 'embarked', 'title']
ord_cols = ['pclass']

# Numerical pipeline: impute → scale
num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler',  RobustScaler()),
])

# Categorical OHE pipeline: impute → one-hot encode
cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
```

```
    ('encoder', OneHotEncoder(drop='first', handle_unknown='ignore',
sparse_output=False)),
])

# Ordinal pipeline: pclass needs no scaling (tree-based) but add for LR
ord_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
])

preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_ohe),
    ('ord', ord_pipe, ord_cols),
])
```

## 8.3 Train & Evaluate Two Models

```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, roc_auc_score,
                             classification_report, ConfusionMatrixDisplay)
import joblib

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest':       RandomForestClassifier(n_estimators=200,
random_state=42),
}

results = {}
for name, model in models.items():
    pipe = Pipeline([('prep', preprocessor), ('model', model)])
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    y_prob = pipe.predict_proba(X_test)[:, 1]
    results[name] = {
        'Accuracy':  accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred),
        'Recall':    recall_score(y_test, y_pred),
        'F1 Score':  f1_score(y_test, y_pred),
        'AUC-ROC':   roc_auc_score(y_test, y_prob),
    }
    print(f'\n=== {name} ===')
    print(classification_report(y_test, y_pred,
target_names=['Died','Survived']))

# Compare results
results_df = pd.DataFrame(results).T.round(4)
print(results_df)

# Export best pipeline
best_pipe = Pipeline([('prep', preprocessor),
                      ('model', RandomForestClassifier(n_estimators=200,
random_state=42))])
best_pipe.fit(X_train, y_train)
joblib.dump(best_pipe, 'titanic_pipeline.pkl')
print('Pipeline saved!')
```

# 9. Hands-On Lab Exercise

> **Dataset:** Use sns.load_dataset('titanic') — no download or login required.

Build a complete, leak-free preprocessing pipeline and train a classifier to predict passenger survival.

## 9.1 Step-by-Step Tasks

1. EDA: Print shape, dtypes, isnull summary, class balance with value_counts(normalize=True)
2. Missing Data: Median impute age; mode impute embarked; drop cabin
3. Outliers: Draw box plots for fare and age; apply Winsorising on fare
4. Feature Engineering: Create family_size and is_alone; extract title from name column
5. Feature Selection: Drop passengerid, name, ticket; plot correlation heatmap
6. Encoding: OHE on sex, embarked, title using ColumnTransformer inside the pipeline
7. Scaling: Use RobustScaler for fare and age; compare with StandardScaler
8. Pipeline: Assemble ColumnTransformer + Pipeline; fit on X_train ONLY
9. Train: Fit Logistic Regression and Random Forest on the pipeline
10. Evaluate: Report Accuracy, Precision, Recall, F1, AUC-ROC for both models
11. Export: Save best pipeline with joblib; reload and predict on X_test[:5]

## 9.2 Deliverables

- Jupyter Notebook (.ipynb) with all steps above
- Markdown cell before each section explaining the WHY behind each choice
- Final performance comparison table (2 models × 5 metrics)
- Saved pipeline file: titanic_pipeline.pkl
- Minimum 3 visualisations: missing-data heatmap, box plots, correlation heatmap

# 10. Assessment Criteria

| Criterion | Description | Marks |
|---|---|---|
| EDA & Missing Data | Correct diagnosis, appropriate imputation strategy, no leakage | 20 |
| Outlier Handling | Detection method justified, Winsorising or RobustScaler applied | 15 |
| Feature Engineering | FamilySize and Title correctly created and validated with group stats | 15 |
| Encoding & Scaling | Correct encoder/scaler chosen per column; inside Pipeline | 15 |
| Pipeline Construction | Leak-free ColumnTransformer + Pipeline; exported .pkl | 20 |
| Model Comparison | Two models trained, 5 metrics reported, best model identified | 10 |
| Code Quality & Docs | Comments, markdown cells, reproducible random_state=42 | 5 |
| **TOTAL** | | **100** |

# 11. Common Mistakes & How to Avoid Them

| Mistake | Correct Approach |
|---|---|
| **Imputing/scaling BEFORE splitting** | Split first → fit imputer/scaler on X_train only |
| **Fitting scaler on full dataset** | scaler.fit(X_train) only; then .transform(X_test) |
| **Label encoding sex for linear models** | Works here (binary) but prefer OHE for multiclass |
| **Dropping all missing rows** | Examine % missing; impute when < 60% |
| **Applying SMOTE to test/val data** | Resample ONLY the training fold — never test |
| **Reporting only accuracy** | Titanic is 62:38 — report Precision, Recall, F1, AUC |
| **Fitting PCA before split** | Fit PCA on X_train; transform X_test separately |
| **Not using random_state** | Set random_state=42 everywhere for reproducibility |

## 12. Quick Reference Cheat Sheet — Titanic

| Problem | Titanic Column | sklearn Solution |
|---|---|---|
| Missing numerical | age | SimpleImputer(strategy='median') |
| Missing categorical | embarked | SimpleImputer(strategy='most_frequent') |
| Drop high-null column | cabin | df.drop(columns=['cabin']) |
| Outlier treatment | fare | mstats.winsorize() or RobustScaler() |
| Feature engineering | sibsp+parch | df['family_size'] = sibsp+parch+1 |
| Title extraction | name | str.extract(r',\s*([^.]+)\.') |
| Binary encoding | sex | LabelEncoder() or OHE |
| Multi-cat encoding | embarked | OneHotEncoder(drop='first') |
| Scale (outliers) | fare, age | RobustScaler() |
| Scale (normal-ish) | age | StandardScaler() |
| End-to-end pipeline | all cols | ColumnTransformer + Pipeline([...]) |
| Export pipeline | — | joblib.dump(pipe, 'model.pkl') |

## 13. References & Further Reading


- Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd ed.). O'Reilly.
- scikit-learn Documentation: https://scikit-learn.org/stable/modules/preprocessing.html
- Titanic Dataset — Kaggle: https://www.kaggle.com/c/titanic
- Seaborn Datasets: https://github.com/mwaskom/seaborn-data
- imbalanced-learn Documentation: https://imbalanced-learn.org/stable/
- Kaggle Learn — Data Cleaning: https://www.kaggle.com/learn/data-cleaning


**Peer Research Lab • Dept. of CSE, BUBT • ML Lab 04**
"Clean data is not an accident — it is the result of intentional preprocessing."